

chapter_6_notes.pdf
PDF

我在复习计算机网络的课程。附件是第6章的slides,, 请你按照课件的思路, 详细的给我过。不要一下子输出所有, 而是输出一节解释, 让我理解一下, 并给我提问的机会。如果我没问题了再继续

已思考 9s >

好, 我们先按课件 **6.1 Introduction & Services** 这一节来过一遍 (只讲这一节, 讲完等你确认没问题再继续)。  chapter_6_notes

6.1 Link layer 在干嘛: 术语 + 位置 + “它负责哪一跳”

课件先把概念钉死 (第 4-5 页):

- **node (节点)**: 主机和路由器都算 node。
- **link (链路)**: 把“相邻两个 node”连起来的通信信道 (有线/无线/LAN)。注意是 **相邻**。
- **frame (帧, 二层包)**: 链路层传输的单位。它把网络层的 **datagram (IP 数据报)** 包进来 (encapsulate)。

一句话: 链路层负责把一个 IP 数据报, 从当前节点送到“物理上相邻的下一个节点”。

(hop-by-hop, 而不是端到端)  chapter_6_notes

为什么强调“每一跳都可能不一样”?

路径上每一段链路可能用不同二层协议 (第 5 页举例: 第一段以太网, 中间可能 Frame Relay, 最后 Wi-Fi)。

所以: 同一个 IP 数据报跨越多跳时, 会被不同的链路层协议反复“装进不同格式的帧里”。

 chapter_6_notes

6.1 Link layer 提供哪些服务 (你要把“菜单”记清楚)

课件把链路层服务分成几类 (第 6-7 页)  chapter_6_notes :

1. Framing (成帧/封装)

把 datagram 放进 frame，并加上 **header/trailer** (例如 MAC 地址、校验信息等)。

2. Link access (共享链路的访问控制)

如果是广播型共享介质 (比如 Wi-Fi、老式同轴以太网)，多台机器要“抢同一条信道”。

这就需要 **MAC 协议** 决定“什么时候谁能发”。帧头里用 **MAC 地址** 标识源/目的 (强调: **MAC ≠ IP**)。  chapter_6_notes

3. Reliable delivery between adjacent nodes (相邻节点可靠交付)

注意是“相邻节点之间”。课件说：

- 在误码率低的链路 (光纤、部分双绞线) **很少用** 链路层可靠机制；
- 无线误码率高，链路层可靠机制更常见。  chapter_6_notes

4. Flow control (流量控制)

控制相邻发送方/接收方节奏，避免接收方被淹没。

5. Error detection / correction (差错检测/纠错)

- detection: 发现错了 (可能丢帧或请求重传)
- correction: 接收方直接把错误位纠正，不用重传

6. Half-duplex / Full-duplex (半双工/全双工)

半双工：两端都能发，但不能同时发；全双工：可以同时收发。  chapter_6_notes

这一页你要形成的“总图景”：

链路层 = 把 IP 数据报装进帧 + (可能) 做差错处理 + (如果共享介质) 协调谁先发 + 把帧交给隔壁那台机器。

6.1 这些功能在哪里实现？ (很关键，决定很多细节)

课件强调链路层通常在 **网卡/NIC (adapter)** 上实现 (第 8-9 页)：  chapter_6_notes

- NIC/芯片实现 **链路层 + 物理层** 的一部分功能；
- 它插在主机总线 (比如 PCI) 上；
- 典型是 **硬件 + 固件 + 驱动软件** 混合实现。

发送端 NIC 做什么？ (第 9 页)

- 从网络层拿到 datagram → **封装成 frame**
- 加上校验位/ (可能的) 可靠传输/流控等

接收端 NIC 做什么？

- 检测错误/ (可能的) 处理可靠传输/流控

- 拆帧取出 datagram → 交给上层（网络层）  chapter_6_notes

小测一下（你答完我就知道你这一节通了没）

1. 你用一句话区分：**datagram** 和 **frame** 分别属于哪一层、负责哪一段传输？
2. 为什么课件反复强调 **MAC 地址** 不同于 **IP 地址**？（提示：作用范围/可移动性）
3. 你觉得“链路层可靠交付”为什么在光纤上很少用、在无线更常用？

你先回我这 3 个点里你最不确定的那一个，我就从那里补齐，然后再进入 6.2（差错检测与纠错）。

Sources



1. datagram属于网络层，负责网络间传输； frame属于链路层，负责相邻物理节点的传输
2. 不懂
3. 光纤误码率低

好，你 1 和 3 基本到位；我们把 **2（MAC vs IP）**彻底讲清楚就能继续。

2) 为什么强调 MAC 地址不同于 IP 地址？

核心就两句话：

- **IP 地址（网络层）**：用来做“跨网络的路由”，回答的是：**这份数据最终要去哪个网络/哪台主机**？
路由器看 IP（更准确：看目的 IP 的前缀）来决定下一跳。
- **MAC 地址（链路层）**：用来做“同一条链路/同一个局域网内的交付”，回答的是：**这一跳我应该把帧交给哪块网卡**？
交换机/网卡在这一跳内靠 MAC 把帧送到正确的网卡口。

你可以把一次端到端传输想成“快递 + 楼内投递”

- **IP 像快递单上的收货地址**（城市/小区/楼号），决定快递在路上怎么走（经过哪些中转站 = 路由器）。
- **MAC 像到达某个中转站后，站内分拣要找的“具体货架/柜子编号”**：只在这一站/这一段链路里有意义。

“每一跳都会换 MAC，但 IP 目标通常不变”

当一个 IP 数据报从 A 到 B：

- IP 头里的 **目的 IP (B) **一般一路不变（除非 NAT 等特殊情况）。
- 但它每过一段链路，都要重新“装进帧”：
 - 这一跳帧的 **目的 MAC = 下一跳那个接口的 MAC**（可能是路由器的某个网口，或者最终主机的网卡）
 - 到了下一跳，旧帧被丢掉，再用新的源/目的 MAC 重新封装继续走

所以课件强调它们不同，是为了让你记住：

IP 解决“跨网怎么走”，MAC 解决“这一跳交给谁”。

用一个最常见的场景帮你落地

你在宿舍电脑访问一个网站（服务器在外网）：

1. 你的电脑要发 IP 数据报：目的 IP = 服务器 IP
2. 但第一跳它其实只能把帧交给“默认网关”（路由器）
 - 所以第一跳以太网帧：目的 MAC = 路由器网口的 MAC
 - 这就是为什么你需要 ARP：用路由器的 IP 问出它的 MAC

快速自检（你只要回答对/不对 + 一句话理由）

- A. “交换机主要看 IP 来转发。”
- B. “路由器在转发时会改帧的 MAC 地址，但一般不改 IP 目的地址。”
- C. “同一个 IP 数据报在不同链路上封装成的 frame 可能格式不同。”

你答完这三句，我们就进入下一节 **6.2 Error detection & correction**。



1. false2. true3. true

全对 ☒ （你已经把“二层看 MAC、三层看 IP、每跳重封装”这条主线抓住了）

那我们进入下一节：**6.2 Error detection & correction（差错检测与纠错）**。

📄 chapter_6_notes

6.2 链路为什么会出错？我们要解决什么问题

链路层面对的是：比特在物理介质上传输时可能被噪声/干扰弄坏（无线更明显）。所以接收方拿到一串比特后，需要回答：

1. 有没有错？（Error detection）
2. 错了能不能直接修？（Error correction）
3. 如果只检测不纠错，错了怎么办？（通常：丢弃 + 让上层或链路层机制重传）

课件这节的重点是：靠冗余 bits（校验冗余）来检测/纠错。 📄 chapter_6_notes

6.2 基本思想：加“冗余位”= 让错误变得“可识别”

发送方把数据（ d bits）变成 codeword（ $d+r$ bits）：

- 多出来的 位是 冗余位（不是“额外信息”，而是用于校验/纠错的结构化约束）
- 接收方收到后检查是否满足这些约束：不满足就判定“有错”

6.2 三种常见方法（你要知道每种“能防什么/不能防什么”）

1) Parity check（奇偶校验，最简单）

- 做法：加 1 bit，使得整串 1 的个数为偶数（或奇数）。
- 能力：能检测 任何奇数个 bit 错误。
- 硬伤：如果发生 偶数个 bit 翻转，可能完全检测不出来（校验位“刚好又对上了”）。

📄 chapter_6_notes

直觉：奇偶校验只在乎“1 的个数的奇偶性”，信息太少，所以会漏掉很多错误模式。

2) Two-dimensional parity（二维奇偶）

- 把数据按矩阵排，行/列都做奇偶校验。
- 能力：比一维强得多；不仅更容易检测，还能在某些情况下定位并纠正单 bit 错误（通过“哪一行哪一列不对”锁定交叉点）。 📄 chapter_6_notes

3) CRC（循环冗余校验，工业界最常用的检测）

- 这是课件后面会重点讲的：把比特串当作多项式，在模 2 除法下算余数，把余数作为校验 bits。
- **能力特点（先给你一张“结论卡”）**：CRC 对“突发错误（burst errors）”检测特别强，工程上普遍用。  chapter_6_notes

这一节你先把“为什么 CRC 强”按直觉记住：它不是只加 1bit，而是加一段结构化冗余；错误一旦打破这种结构，余数就对不上。

小测：确保你跟上（只答对/错 + 1 句理由）

1. 一维奇偶校验：能检测 3 个 bit 翻转，但可能检测不出 2 个 bit 翻转。
2. 二维奇偶：如果只有 1 个 bit 出错，理论上可以定位到那一位。
3. CRC 的目标主要是“纠错”，不是“检测”。

你答完这 3 个判断题，我就顺着课件把 **CRC 的“模2除法/生成多项式/突发错误检测能力”**按课件推导讲下去。

Sources



这些校验方法都给我展示一下计算过程

已思考 1m 59s >

行，我们把课件里常见的三类都各做一个“手算示例”：**1-bit parity**、**2D parity**、**CRC**（过程按课件思路：加冗余位→接收端检验）。  chapter_6_notes

1) 一维奇偶校验（Parity bit）怎么算

假设用 **偶校验 even parity**：让“整串 1 的个数”为偶数。

例子：数据位

D = 1011001

数 1 的个数：1 有 4 个（偶数）

所以校验位 $p = 0$ （加 0 仍是偶数）

发送的比特串：

D || p = 1011001 0

接收端怎么检验：收到 10110010，再数 1 的个数是否为偶数。若变成奇数 $\Rightarrow$ 判定出错。

 chapter_6_notes

2) 二维奇偶校验（Two-dimensional parity）怎么算 + 怎么定位单比特错

把数据按矩阵摆放，对 **每一行、每一列**都做偶校验。

例子：16 个数据位（4×4）

D = 1101 0010 0111 0100

摆成矩阵（每行 4 位）：

makefile

 Copy code

Row1: 1 1 0 1

Row2: 0 0 1 0

Row3: 0 1 1 1

Row4: 0 1 0 0

(a) 行校验位怎么求

对每行做偶校验（让该行 1 的个数为偶数）：

- Row1 1101 有 3 个 1（奇数） $\rightarrow$ 行校验位 = 1
- Row2 0010 有 1 个 1（奇数） $\rightarrow$ 行校验位 = 1
- Row3 0111 有 3 个 1（奇数） $\rightarrow$ 行校验位 = 1
- Row4 0100 有 1 个 1（奇数） $\rightarrow$ 行校验位 = 1

所以行校验列是：[1,1,1,1]

(b) 列校验位怎么求

对每列做偶校验：

- Col1: 1,0,0,0 有 1 个 1 → 列校验位 1
- Col2: 1,0,1,1 有 3 个 1 → 列校验位 1
- Col3: 0,1,1,0 有 2 个 1 → 列校验位 0
- Col4: 1,0,1,0 有 2 个 1 → 列校验位 0

所以列校验行是： [1,1,0,0]

(有的实现还会再加右下角一个“校验位的校验”，这里先不强求。)

(c) 为什么能定位“单比特错误”

假设传输中 **Row3 Col2** 那一位翻转 (原来是 1 变 0)。

接收端重新算行/列校验，会发现：

- 只有 **第 3 行** 的奇偶性不对
- 只有 **第 2 列** 的奇偶性不对

交叉点 (Row3, Col2) 就是出错位 → 把那一位翻回去即可纠正单比特错。

 chapter_6_notes

3) CRC 怎么算 (模2除法) ——完整长除示例

CRC 关键点：“减法”是模 2，所以就是 **XOR**，没有借位/进位。  chapter_6_notes

选一个小例子 (好手算)：

- 数据 $D = 1101011011$
- 生成多项式 (除数) $G = 10011$
其长度是 5，所以 $r = 4$ (要加 4 个 CRC 位)

(1) 先在数据后面补 r 个 0

yaml

 Copy code

$D \cdot 2^r = 1101011011\ 0000$

(2) 用 G 做“模2长除”，得到余数 R (4 位)

下面我只列出“需要 XOR 的那些步” (遇到当前位是 0 就跳过)：

初始:

 Copy code

11010110110000

步1: 对齐到最左边 (位置0), 取前5位 11010

vbnet

 Copy code

11010 XOR 10011 = 01001

更新后整体变为:

 Copy code

01001110110000

步2: 移动到位置1, 当前前导位为1, 取 10011

vbnet

 Copy code

10011 XOR 10011 = 00000

更新后:

 Copy code

00000010110000

步3: 移动到位置6, 取 10110

vbnet

 Copy code

10110 XOR 10011 = 00101

更新后:

 Copy code

00000000101000

步4：移动到位置8，取 10100

```
vbnet
```

[Copy code](#)

```
10100 XOR 10011 = 00111
```

更新后：

[Copy code](#)

```
00000000001110
```

最后 余数 R = 最右边 4 位 = 1110。

(3) 发送码字 = 原数据 + 余数

```
ini
```

[Copy code](#)

```
T = D || R = 1101011011 1110
```

接收端检验：把 T 再除以 G，若余数是 0000 则通过；否则判错。 [chapter_6_notes](#)

你想先把哪一个练熟一点？我可以给你出一道“你来手算”的小题（Parity / 2D parity / CRC 三选一），你算我来逐步对答案。

[Sources](#)

[Copy](#) [Like](#) [Quote](#) [Share](#) [Refresh](#) [More](#)

[继续过课件](#)

已思考 17s >

好，我们继续按课件走到 **6.3 Multiple access links, protocols**（多路访问链路与MAC协议）的开头部分：先把“问题是什么 + 协议分类 + 理想目标 + 最第一类方案（信道划分 TDMA/FDMA）”讲清楚。讲完我停下来让你提问，再继续到 ALOHA/CSMA。

[chapter_6_notes](#)

6.3 这节在解决什么问题：广播链路上的“谁先说”

课件先区分两类链路：  chapter_6_notes

- **point-to-point (点到点)**：一条链路就两端（例如主机↔交换机端口、PPP拨号）。基本不存在“多人抢话筒”问题。
- **broadcast (广播/共享介质)**：一条共享信道上挂很多节点（老式以太网总线、上行HFC、Wi-Fi）。这时如果两个节点同时发，信号叠加就会**collision (碰撞)**。

 chapter_6_notes

所以 **multiple access protocol / MAC 协议** 就是一个分布式算法，决定：每个节点什么时候可以发，并且协调信息也必须走这条共享信道（没有“额外的协调通道”）。

一个“理想的MAC协议”长什么样

给定广播信道速率 **R bps**，课件列了四条理想性质：  chapter_6_notes

1. 只有 1 个节点想发时，它能以 **R** 的速率独占发送
2. 有 **M** 个节点都想发时，每个节点平均能拿到 **R/M**
3. **完全去中心化**：没有主控节点、也不需要全局时钟同步/时隙同步
4. 简单

这四条其实彼此“拉扯”：比如想要(3)去中心化又想要(2)公平分配，经常会导致冲突/开销。

MAC 协议三大类 (taxonomy)

课件把 MAC 分成三类：  chapter_6_notes

1. **channel partitioning (信道划分)**：把信道切成小块（时间/频率/码），每人拿一块独占
2. **random access (随机接入)**：不切，允许碰撞；碰撞后再“恢复”（重传/退避）
3. **taking turns (轮流)**：显式轮流，谁有更多数据就多占一点轮次

今天这一步我们只讲第 1 类：信道划分（TDMA/FDMA）。随机接入（ALOHA/CSMA）下一步讲。

6.3.1 信道划分：TDMA / FDMA

TDMA (Time Division Multiple Access)

思路：时间被划成一轮一轮的 **slot**，每个节点每轮固定一个slot。  chapter_6_notes

- 优点：不会碰撞；公平、可预测
- 缺点（课件强调的痛点）：如果某个节点这一轮没数据，它的 slot 就空转浪费 (“unused slots go idle”)。  chapter_6_notes

FDMA (Frequency Division Multiple Access)

思路：把频谱切成若干频带，每个节点固定占一个频带。  chapter_6_notes

- 优点：同样不会碰撞；也很稳定
- 缺点：如果节点没数据，它的频带那段时间也闲着浪费。  chapter_6_notes

这就是“信道划分”的典型 trade-off：

用“零碰撞 + 简单公平”换来“低负载时的利用率浪费”。

快速自检（你回答我就知道你跟上没）

1. TDMA/FDMA 为什么在“低负载”时可能效率不高？（一句话）
2. 课件理想 MAC 的第 3 条“fully decentralized”意味着什么？（别用术语，用白话）

你答完我们就继续课件下一段：**random access (Slotted ALOHA 开始)**。

Sources



1. 会有空闲2. 没有主控节点

已思考 11s >

好，继续按课件走：进入 **Random access protocols (随机接入协议)** → **Slotted ALOHA** → **Pure ALOHA** → (引出) **CSMA**。我先把 ALOHA 这一小段讲透，讲完我就停，让你提问；你说 OK 再继续 CSMA/CSMA-CD。  chapter_6_notes

6.3 Random access 的基本设定：允许撞车，再想办法恢复

随机接入的共同点是：**不预先把信道切块**，谁有包就尝试以全速发；如果多人同时发就会 **collision**，然后协议要规定：

- 怎么检测 collision
- 怎么从 collision 恢复（通常是延迟重传/退避）  chapter_6_notes

6.3.2 Slotted ALOHA（有时隙的 ALOHA）

(1) 课件的假设（很关键）

Slotted ALOHA 的假设是：  chapter_6_notes

- 所有 frame 同样大小
- 时间被切成等长 **slot**（一个 slot 刚好够发 1 个 frame）
- 节点只能在 **slot 开始时刻**发
- 节点之间 **时间同步**
- 若同一 slot ≥ 2 个节点发送，则大家都能检测到碰撞

(2) 运行规则（operation）

当某节点拿到一个新 frame：

- 它会在下一个 **slot**开始时发送
- 如果这次 slot 没碰撞：下一个 slot 可以继续发新 frame
- 如果碰撞：之后每个 slot 以概率 重传，直到成功  chapter_6_notes

(3) 优缺点（课件的 Pros/Cons）

****Pros：**单节点活跃时可连续满速；较去中心化（只需要 slot 同步）；简单。

****Cons：**仍会碰撞浪费 slot；也会有空 slot；还需要时钟同步。  chapter_6_notes

Slotted ALOHA 的效率推导（课件的重点公式）

设：有 个节点都“总有包要发”，每个节点在某个 slot 里以概率 发送。

 chapter_6_notes

(1) “某个指定节点”在一个 slot 成功的概率

它要成功必须：

- 它发：概率

- 其他 个都不发：概率

所以：

📄 chapter_6_notes

(2) “整个系统”在一个 slot 有成功的概率 (=效率的核心)

一个 slot 里“恰好有一个节点发”才算有效利用 (成功 slot)：

📄 chapter_6_notes

课件把 **efficiency** 定义为：长期来看成功 slot 的比例 (也就是信道用于“有用传输”的时间比例)。

📄 chapter_6_notes

(3) 最大效率结论

把 对 取最优 ，再让 ，课件给出经典结论：

—

也就是说：再理想也只有 37% 的时间在成功传输 (其余是空 slot 或碰撞)。

📄 chapter_6_notes

6.3.3 Pure (Unslotted) ALOHA (无时隙 ALOHA)

Slotted ALOHA 要同步；Pure ALOHA 不要同步，更“随性”：frame 一到就立刻发。

📄 chapter_6_notes

(1) 为什么碰撞概率更大：vulnerable period 变长

课件的描述是：如果某帧在 时刻发送，它会和其它在区间

内发送的帧发生碰撞 (这里的 1 是“一个 frame 的发送时间单位”)。

也就是：易受碰撞的时间窗是 2 个 frame 时间。

📄 chapter_6_notes

(2) 效率结论更差

同样做概率推导，课件给出：

并在取最优后得到：

比 slotted 的 0.37 还差一半。  chapter_6_notes

到这里你需要牢牢记住的“考点句”

- **slotted ALOHA**：需要同步；最大效率 0.37 。  chapter_6_notes
- **pure ALOHA**：不需要同步；易受碰撞时间窗翻倍；最大效率 0.185 。
 chapter_6_notes
- 它们都属于“随机接入”：低负载时爽（一个人几乎独占），高负载时碰撞开销爆炸。
 chapter_6_notes

你先回答两个超短自检（对/错 + 一句话）

1. Pure ALOHA 的 vulnerable period 比 slotted ALOHA 长一倍，所以最大效率更低。
2. slotted ALOHA 的浪费来源只有 collision，没有 idle slot。

你答完我就继续讲 **CSMA**（先听后说）→ 为什么仍会撞（传播时延）→ **CSMA/CD** + 二进制指数退避（以太网）。  chapter_6_notes

Sources



1. false? 2. no

来，我们先把你这两个自检纠正一下，然后立刻进入下一小节 **CSMA**（Carrier Sense Multiple Access）。  chapter_6_notes

先纠正自检

1) “Pure ALOHA 的 vulnerable period 比 slotted ALOHA 长一倍，所以最大效率更低。”

这是 true 

原因：Pure ALOHA 里一帧会被“前后各 1 个帧时长”的其它发送撞到，所以易受碰撞窗口是 2 个帧时长；而 slotted 把发送对齐到 slot 开始，把有效碰撞窗口压小，所以效率更高（课件结论：slotted 上限 $\frac{1}{2}$ ，pure 上限 $\frac{1}{4}$ ）。

2) “slotted ALOHA 的浪费来源只有 collision，没有 idle slot。”

这是 false 

slotted 里如果某个 slot 没人发，那就是 idle slot（也算浪费）。  chapter_6_notes

继续课件：CSMA（先听后说）

ALOHA 最大问题：完全不管别人说没说，所以撞得很惨。

CSMA 的直觉改进：发送前先“载波侦听”信道是否空闲（listen before talk）。

 chapter_6_notes

1) CSMA 的基本规则（核心一句话）

- 信道忙：就先等（defer）
- 信道闲：就发（transmit）  chapter_6_notes

听起来好像能避免碰撞，但课件马上说：仍然会碰撞。关键原因是 传播时延 propagation delay。  chapter_6_notes

2) 为什么“先听后说”仍会撞：传播时延导致“听到的不是实时状态”

想象两个相距很远的节点 A 和 B：

- A 在时刻 t 听到信道空闲 → 开始发
- 由于信号传播需要时间，B 在 $t + \tau$ 也去听，可能仍然听到空闲（因为 A 的信号还没传播到 B）
- B 也开始发 → 两者的信号在链路中间相遇 → collision  chapter_6_notes

这就是课件要你记住的：

CSMA 减少碰撞，但无法消灭碰撞；传播时延越大，越容易发生这种“同时起步”。

3) CSMA 的几种“坚持策略” (persistence)

课件通常会提到三种常见策略（你记住“行为差异”就行）： chapter_6_notes

- **1-persistent CSMA**：一旦侦听到空闲就立刻发（很激进，空闲一出现大家可能一起冲）
 - **non-persistent CSMA**：忙就随机等一会再听（更温和，减少同时起步）
 - **p-persistent CSMA（用于时隙信道）**：空闲时以概率 p 发，否则等下个 slot（折中）
- （这里的细节一般不要求你推公式，但要会解释它们在“碰撞 vs 延迟”上的取舍。）

再继续一点：CSMA/CD（以太网经典）

在 有线以太网里，除了“先听后说”，还可以 边发边听：

- 如果发着发着发现信道上出现了“和自己不一致的信号” → 说明撞了
- 立刻停止发送（而不是把整帧浪费完）

这叫 **Collision Detection (CD)**，合起来就是 **CSMA/CD**。 chapter_6_notes

Binary Exponential Backoff（二进制指数退避）直觉

碰撞后不能马上再冲，否则又撞。以太网用退避：

- 第 1 次撞：在 一个“基本时隙”里随机等一个
- 第 2 次撞：在 里随机等一个
- 第 k 次撞：范围变成 （到某个上限截断） chapter_6_notes

直觉：越拥堵越“拉开间隔”，系统更容易稳定下来。

停一下给你提问（你答完我再继续后面的图/效率讨论）

1. 你能用自己的话复述：**CSMA** 仍会碰撞的根本原因是什么？
2. 你觉得 **CSMA/CD** 相比纯 **CSMA** 的关键收益是什么（从“浪费时间”角度说）？

Sources



1. 传播时延 2. 边发边听，能够及时停止发送被撞的 frame

完全正确 ✅ （你已经抓住这段最核心的两点）

那我们继续把课件这部分收尾：为什么 CSMA 的效率通常比 ALOHA 好、以及 CSMA/CD 里的“最小帧长/slot time”为什么存在（这块经常考概念题）。

1) CSMA 为什么通常比 ALOHA 效率高

对比 ALOHA：

- ALOHA：不看信道状态，空闲也可能浪费成 idle、忙也照样撞
- CSMA：忙的时候先不发 → 大量“明知会撞的发送”被避免了，所以碰撞概率明显下降（尤其在传播时延不大的 LAN 里）。 [📄 chapter_6_notes](#)

但注意：传播时延越大（信道越“长”），CSMA 越接近 ALOHA 的糟糕状态，因为“听到空闲”越来越不可靠。课件通常会用一个比值来表达直觉：

a 越小（传播时延相对帧发送时间越小），CSMA 越好。 [📄 chapter_6_notes](#)

2) CSMA/CD 的“关键工程点”：必须保证能检测到碰撞

你刚说了“边发边听及时停”，但这里有一个隐藏前提：发送方在还没发完这一帧之前，必须有机会检测到碰撞。 [📄 chapter_6_notes](#)

为什么需要“最小帧长”

考虑最坏情况：A 和 B 在链路两端几乎同时开始发。

碰撞信号从远端传播回来要时间，最坏需要一个“往返传播时间”约 才能让最先开始发的人确认碰撞。

如果某个站点发的帧太短，可能出现：

- 它把整个帧都发完了（以为成功），然后才“听到”碰撞信息
- 这就等于碰撞没被及时检测，机制失效

所以以太网规定：帧的发送时间必须 $\geq$ 。换句话说：

这就是 **slot time / minimum frame length** 的来源。  chapter_6_notes

你可以把它记成一句考点：

以太网最小帧长不是“为了效率”，而是为了让 CSMA/CD 碰撞检测在最坏情况下仍然来得及发生。

3) Binary Exponential Backoff 再补一句“为什么指数”

你知道规则了，这里补一个考试友好的直觉：

- 碰撞说明“同一时刻有多个人抢”
- 如果大家都只做固定的小随机等待，拥堵时仍容易重复同步再撞
- 指数扩大窗口让“随机化尺度”随拥堵快速增长 → 更快打散同步性 → 更容易稳定

 chapter_6_notes

到这里停一下（确认你把这段“最小帧长”吃透）

请你判断并解释一句话（对/错 + 为什么）：

“如果把以太网的最小帧长限制取消，CSMA/CD 仍然能正常工作，只是效率变差。”

你答完我就继续课件下一节：**Taking turns protocols (Polling / Token passing)** 或者直接转到 **LANs (Ethernet) 结构**（看你课件顺序）。

Sources



你忽略了NIC

你说得对：前面我把 MAC/ALOHA/CSMA 当成“抽象协议”在讲了，但课件这一章其实一直在强调 **链路层很多机制是由 NIC（网卡/adapter）来实现的**，不是靠应用/进程去“手动执行协议”。我把这块补齐，并把它和你刚学到的 **CRC / CSMA/CD / backoff** 对上号。

 chapter_6_notes

NIC 在这一章里到底扮演什么角色

1) NIC 是链路层的“执行者”

链路层的 framing、MAC 地址处理、差错检测（CRC）、以及共享介质上的接入控制（比如以太网 CSMA/CD）——大多都在 NIC 的硬件/固件里跑，OS 通过驱动配置它、喂数据给它、从它拿数据。  chapter_6_notes

你可以把它理解成：

主机/OS 负责“要发什么”（IP 数据报），NIC 负责“怎么在这条链路上发出去并交付到下一跳”（帧 + MAC + 介质访问 + 差错处理）。

2) 发送路径：datagram → NIC → frame → 线上比特流

典型流程（抽象到考试所需程度）：

1. 网络层把 IP datagram 交给链路层接口（驱动）
2. NIC 封装成 frame：加上目的/源 MAC、type/length、以及 trailer 里的 CRC

 chapter_6_notes

3. 如果是共享介质（经典以太网）：NIC 负责 CSMA/CD

- carrier sense（先听是否空闲）
- transmit（空闲就发）
- collision detect（边发边听，撞了就知道）
- jam + abort（撞了发 jam 信号/停止）
- binary exponential backoff（退避后重传）  chapter_6_notes

4. NIC 把 frame 变成物理层信号发到链路上

所以我们讲的“随机退避/重传”，在以太网里就是 NIC 在做；CPU 不会一比特一比特盯着线。

3) 接收路径：线上比特流 → NIC → frame → datagram

1. NIC 从物理层收到比特，重组为 frame
2. 硬件做 CRC 校验：错了通常直接丢弃（不会交给 IP）  chapter_6_notes
3. 正确则 拆帧取出 datagram，交给主机网络层（驱动/内核协议栈）

4) NIC 和你刚问的“最小帧长 / slot time”怎么对应

这条约束的根因就是：要让 NIC 的 collision detection 来得及发生。

- CSMA/CD 要求：NIC 在“还没发完这一帧”时，最坏情况下也能听到碰撞（大致需要往返传播时间）。
- 如果帧太短，NIC 可能“发完了才听到碰撞”，那对它来说就像成功了一样——CSMA/CD 机制就失效。  chapter_6_notes

所以：最小帧长不是抽象协议的要求，是 NIC 能否正确执行 CSMA/CD 的工程前提。

你现在可以用一句话把 NIC 的定位记住

“链路层协议（成帧、MAC、CRC、CSMA/CD、退避）在实现上主要由 NIC 执行；OS/驱动负责把 datagram 交给 NIC，并接收 NIC 通过校验后的 datagram。”

你想不想我接着按课件把 **CSMA/CSMA-CD 那几张图（碰撞发生的时间线、以及 backoff 的窗口增长）**再走一遍？如果你说继续，我就从那一页开始往下讲。

Sources



课件的how long should the nic wait

已思考 20s >

课件这句 “How long should the NIC wait?” 指的是：在 CSMA/CD 里，一旦 NIC 在发送过程中检测到 collision 并中止发送后，它不能立刻重发，而是要按 Binary Exponential Backoff（二进制指数退避）随机等待一段时间再回到“先听再发”。  chapter_6_notes

课件给的精确定义

- 第 m 次碰撞之后（mth collision），NIC 随机选一个整数

- 然后等待

也就是等 K 个“发送 512 比特所需的时间”。  chapter_6_notes

等待结束后，NIC 回到算法第 2 步：继续 carrier sense（听信道忙/闲），闲就发，忙就等到闲再发。  chapter_6_notes

你可以用几个 m 的例子快速记住

- 第 1 次撞 ($m=1$): $K \in \{0,1\} \rightarrow$ 等 0 或 1 个“512-bit time”
 - 第 2 次撞 ($m=2$): $K \in \{0,1,2,3\} \rightarrow$ 最多等 3 个单位
 - 第 3 次撞 ($m=3$): $K \in \{0\dots7\} \rightarrow$ 最多等 7 个单位
- => 撞得越多，随机等待窗口指数变大（课件那句 “longer backoff interval with more collisions”）。  chapter_6_notes

为什么单位是 “512 bit times”

课件这里把它当作以太网的一个**基本时隙单位 (slot time)**：用“发 512 比特所需时间”做粒度，方便表达退避时间；同时它也和以太网里“碰撞检测必须在一定时间内发生（与传播时延相关）”那套工程约束绑定在一起（你后面如果讲到最小帧长/slot time，会更顺）。

 chapter_6_notes

如果你愿意，我们下一步就把它和紧接着那页 **CSMA/CD efficiency** 里 $t_{\text{prop}} / t_{\text{trans}}$ 的关系连起来讲（为什么传播时延越小越好）。  chapter_6_notes

Sources



继续

已思考 16s >

继续：我们现在严格跟着课件到 **Link Layer 6-36: CSMA/CD efficiency** 这一页，把它讲透（讲完我停一下给你提问）。  chapter_6_notes

CSMA/CD efficiency 这一页到底在说什么

课件先定义两个时间量：  chapter_6_notes

- τ : LAN 上两节点之间**最大传播时延** (max propagation delay)
- T_{fr} : 发送一个**最大长度帧**所需的时间 (time to transmit max-size frame)

然后给两个极限直觉：  chapter_6_notes

- τ 小时, 效率 ≈ 1 (几乎不会“同时起步”导致碰撞)
- T_{fr} 小时, 效率 ≈ 1 (帧很长, 传播时延那点“起步混乱”相对开销很小)

关键公式 (课件右下角)

课件给一个近似式：  chapter_6_notes

$$\eta \approx \frac{1}{1 + 2\tau/T_{fr}}$$

你可以把它读成一句话：

效率取决于比值 $\frac{T_{fr}}{\tau}$ 。

LAN 越“短” (传播时延小) / 帧越“长” (发送时间大), $\frac{T_{fr}}{\tau}$ 越小, 效率越接近 1。

“为什么是 5a?” (你只需要抓住课件想要的直觉)

课件不是要你背推导细节, 而是要你理解: 在 CSMA/CD 里, 即便“边发边听”能尽早停止碰撞, **每次发生竞争/碰撞仍然会浪费一个与传播时延同量级的时间** (因为“你得等信号传播开, 大家才知道该停/该退避”)。

所以在高负载时, 平均每次成功发送前, 会累计若干个“与 τ 同量级”的浪费; 课件用一个常数 (经验近似) 把它写成 5τ , 于是就出现了 $5a$ 。  chapter_6_notes

你可以立刻做一个“数量级判断”

- 若 $\tau < 100 \text{ ns}$ (很高)
- 若 $\tau > 100 \text{ ns}$ (掉很厉害)

这也解释了: **CSMA/CD 更适合“短距离、低传播时延”的有线 LAN。**  chapter_6_notes

小停一下 (确认你真的吃透这页)

1. 这页里，什么情况下效率会更高：**LAN 更长**还是 **LAN 更短**？为什么（用 说）
2. 同一个 LAN 里，把“平均帧长”变长，一般会让效率更高还是更低？为什么（用 说）

你答完我就继续下一节 “Taking turns” protocols: polling / token passing（课件 6-38/6-39）。  chapter_6_notes

Sources



更短；变高

已思考 11s >

继续，按课件 **Link Layer 6-37~6-39: Taking turns MAC protocols（轮流型 MAC）** 这一小节走。  chapter_6_notes

6-37：为什么会有 “taking turns”（想要“两全其美”）

课件先回顾前两类的硬伤：  chapter_6_notes

- **channel partitioning (TDMA/FDMA)**：高负载时“公平且效率高”，但**低负载**时很蠢：即使只有 1 个活跃节点，也只能拿到 $1/N$ 的带宽，还要等轮到自己（delay in channel access）。
- **random access (ALOHA/CSMA)**：低负载时很爽（一个人可独占信道），但**高负载**时碰撞开销很大（collision overhead）。

于是第三类 **taking turns** 的目标就是：

低负载时尽量让活跃节点快速用上信道；高负载时尽量避免碰撞并保持公平。

 chapter_6_notes

6-38：Polling（轮询）

课件定义：有一个 **master** 节点，按顺序“点名”邀请各个 **slave** 发送。  chapter_6_notes

你可以把它想成课堂点名发言

- master 发一个 **poll**: “A 你说”
- A 回 **data** (或者说“我没东西”)
- master 再 poll 下一个

课件列的 concerns (你要会解释每个)

- **polling overhead**: 点名本身要占信道 (poll 帧/控制开销)。  chapter_6_notes
 - **latency**: 即使你现在有数据, 也可能要等 master 点到你才轮到你说。
 chapter_6_notes
 - **single point of failure (master)**: master 挂了, 整个系统就停了。  chapter_6_notes
 - 课件还提一句: 常用于 “dumb” slave devices (从设备很简单、由中心统一调度的场景)。  chapter_6_notes
-

6-39: Token passing (令牌传递)

课件定义: 一个 **token** (令牌) 在节点之间按固定顺序传。只有拿到 token 的节点才允许发数据。  chapter_6_notes

你可以把它想成“传话筒”

- 话筒 (token) 传到你手上:
 - 你有数据就发 (data)
 - 没数据就把 token 继续传下去 (nothing to send)

课件列的 concerns

- **token overhead**: token 本身就是控制消息, 占用带宽。  chapter_6_notes
 - **latency**: 你要等 token 传到你这 (尤其节点很多时)。  chapter_6_notes
 - **single point of failure (token)**: token 丢了/坏了, 大家都发不了 (需要额外机制恢复 token)。  chapter_6_notes
-

小结: Polling vs Token passing (你用一句话区分就够)

- Polling: 中心化点名, 怕 master 挂;
 - Token: 去中心化一些 (没有 master), 但怕 token 出故障/丢失。  chapter_6_notes
-

到这先停一下 (两道自检)

1. 为什么 taking-turns 在高负载下通常比 random access 更稳？（用“碰撞”回答）
2. polling 的 single point of failure 是谁？ token passing 的 single point of failure 是谁？

你答完我们就继续课件下一页：**Cable access network case study (HFC/DOCSIS)** 开始。  chapter_6_notes

Sources



1. 避免碰撞和重传 2. master, token

已思考 21s >

继续：我们现在按课件 **Link Layer 6-40~6-41: Cable access network (HFC/DOCSIS 案例)** 这一小节走一遍（讲完我停一下给你提问）。  chapter_6_notes

6-40 Cable access network: 拓扑 + 上下行“天然不对称”

课件这页先把 cable 上网的结构摆出来：****cable head end（局端）****里有 **CMTS (cable modem termination system)**，一端连 ISP/Internet，另一端连很多住户的 cable modem。  chapter_6_notes

然后给两个关键数字/结论：  chapter_6_notes

- **downstream（下行）**：多个 **40 Mbps** 的频道（broadcast），本质上是 **CMTS 往所有住户广播发**
- **upstream（上行）**：多个 **30 Mbps** 的频道，但这里是 **multiple access**：很多用户要“共享上行”，会在某些上行时隙里竞争（contend），另一些时隙是被分配的（assigned）

直觉：下行像“电视台播出”（一个源发、很多人收）；上行像“很多人往局端回传”（多源抢同一资源）。

6-41 DOCSIS: 把三类 MAC 思想“混合”起来用

这一页讲 DOCSIS（有线宽带标准）的 MAC 设计：它不是纯 TDMA/纯随机接入，而是混合。  chapter_6_notes

1) 频域：FDM 划分上下行频道

- **FDM over upstream, downstream frequency channels**：上下行都在不同频带里分多个频道（你可以理解为：不同频点承载不同下行/上行数据流）。  chapter_6_notes

2) 上行时间域：TDM + “一部分分配、一部分竞争”

课件一句话很关键：  chapter_6_notes

- **TDM upstream: some slots assigned, some have contention**

也就是：上行被切成很多很小的 **minislots**（迷你时隙），其中：

- **assigned minislots**：CMTS 明确“分配给某个 cable modem 用来发数据帧”
- **contention minislots**：大家可以在这里“抢”着发 **request**（请求）帧

3) MAP frame：下行广播“上行排班表”

课件写：**downstream MAP frame: assigns upstream slots**。  chapter_6_notes

意思是：CMTS 会在下行发一个 **MAP frame**，告诉大家在时间区间 内：

- 哪些 minislots 被分配给谁来发上行 data frame
- 哪些 minislots 留作 request contention

你可以把 MAP 理解成“下一段时间的上行时间表”。

4) Request 怎么抢：随机接入 + 二进制退避

课件写得很直接：  chapter_6_notes

- request for upstream slots transmitted **random access (binary backoff)** in selected slots

也就是：在 contention minislots 里，大家用随机接入方式发 request；如果撞了，就用 **binary backoff**（和你刚学的以太网 NIC backoff 是同一个思想）。

这套设计为什么合理（把三类 MAC 的优缺点拼起来）

- 纯随机接入：上行高负载会撞爆
- 纯 TDMA：低负载时大量空转，而且请求/突发流量不灵活
- DOCSIS：用“**竞争(request) + 分配(data)**”的组合
 - request 小、偶发，撞了代价也小

- 真正的大数据帧尽量走 assigned minislots, 避免碰撞浪费

这就是课件想让你体会的“best of both worlds”落地方式。  chapter_6_notes

小停一下（两问就够）

1. MAP frame 的作用你能用一句话说清楚吗？
2. 为什么把“竞争”主要放在 request 而不是直接让 data frame 也去竞争？

你答完我就继续下一页 6-42 Summary of MAC protocols, 然后进入 6.4 LANs: MAC addressing & ARP。  chapter_6_notes

Sources



1. ? 2. ?

没问题，这两问正好是这一页的核心。我们把它讲清楚，然后继续到 6-42 总结页。

 chapter_6_notes

1) MAP frame 是干嘛的？（一句话版 + 稍微展开）

一句话：MAP frame 是 CMTS 在下行广播的“上行时隙排班表/时间表”，告诉每个 cable modem：接下来哪些 upstream minislots 谁能用、哪些是用来大家抢着发 request 的。

 chapter_6_notes

展开成你能复述的流程就是：

- 上行被切成很多 minislots（小时间片）
- CMTS 定期下发 MAP：
 - 指定 “modem A 在 slot 10–12 发数据”
 - “slot 1–3 是 contention, 大家可以来发 request”  chapter_6_notes

2) 为什么让“竞争”主要发生在 request, 而不是 data frame?

因为 **data frame 很大、很贵**；**request 很小、很便宜**。

- 如果让 data frame 直接随机接入去抢：一旦碰撞，你浪费的是“一整个大帧的发送时间”，代价巨大 → 高负载时效率崩。
- 现在改成：先在 contention minislots 里抢着发 **request**（小包，撞了损失小），抢到了之后 CMTS 再通过 MAP 分配一段专用 **slots** 给你发真正的 data frame，这样 **数据尽量不碰撞**。  chapter_6_notes

你可以把它理解成：

先“排队领号”（request 竞争），再“叫号去窗口办理大业务”（data 分配传输）。

继续过课件：6-42 Summary of MAC protocols（这页你要形成一张总对照表）

这一页是在收束 6.3：把三类 MAC 协议的适用场景一口气对齐。  chapter_6_notes

Channel partitioning (TDMA/FDMA)

- 优：无碰撞、可预测、公平
- 劣：低负载浪费空闲（slot/频带闲着）  chapter_6_notes

Random access (ALOHA / CSMA)

- 优：低负载下效率高、简单、去中心化
- 劣：高负载碰撞开销大（重传/退避带来浪费和时延）  chapter_6_notes

Taking turns (Polling / Token)

- 优：高负载下可避免碰撞，公平性好
- 劣：控制开销 + 等待带来的时延；还存在单点故障（master / token）

 chapter_6_notes

DOCSIS（电缆上网）

- 混合：FDM + 上行 minislots，request 竞争、data 分配（你刚刚理解的那套）

小停一下（确认你“总表”脑子里有了）

你用一句话回答：

如果“用户很少时也要尽量别浪费带宽”，但“用户很多时又不想撞得太狠”，你会倾向哪一

类？为什么？

你答完我就进入课件下一大节 **6.4 LANs: MAC addresses & ARP**（这部分会把之前问的 next-hop / MAC / ARP 串起来）。

Sources



1. taking turns & docsis

已思考 15s >

好，继续按课件 **6.4 LANs: addressing, ARP** 这一节来（我先讲 **MAC 地址 + ARP（同一 LAN）+ ARP表**，讲完停下来给你提问；你 OK 了我再继续讲后面的“跨LAN经由路由器的寻址walkthrough”）。  chapter_6_notes

6-44~6-46：MAC 地址到底是什么，和 IP 的关系

课件对比得很直接：  chapter_6_notes

- **IP 地址（32-bit）**：网络层地址，用于 **三层转发/路由**（跨网络）。
- **MAC 地址（48-bit）**：链路层地址，用于“本地”把 **frame** 从一个接口送到同一网络里的另一个物理相连接口（同一 LAN / 同一子网意义下）。
 - 通常 **烧录在 NIC ROM**（也可能软件可改）
 - 常用 **16 进制表示**：如 **1A-2F-BB-76-09-AD**（每个十六进制字符=4 bits）

 chapter_6_notes

- **每个 NIC 在一个 LAN 上的 MAC 唯一**（“网卡级别”的唯一标识）  chapter_6_notes

6-46：为什么 MAC “可搬家”，IP 不行

- MAC 是 **flat address**（扁平、不含拓扑层次）→ 网卡插到另一个 LAN 也不需要改“地址本身”，所以说 **portability**。
- IP 是 **hierarchical**（带子网前缀，地址依赖你接入哪个子网）→ 换网通常要换 IP。

 chapter_6_notes

6-47~6-49: ARP (Address Resolution Protocol) 在同一LAN怎么把 IP 变成 MAC

问题

“我知道对方的 IP，但我要发二层帧，必须填 **目的 MAC**。那目的 MAC 怎么来？”——这就是 ARP。  chapter_6_notes

ARP 同一LAN的基本流程（课件图就是这三步）

假设 A 要给同一 LAN 内的 B 发 IP 数据报，但 A 的 ARP 表里没有 B 的 MAC：

 chapter_6_notes

1. A 广播 ARP Query（问：谁是这个 IP？）

- 二层目的 MAC = FF-FF-FF-FF-FF-FF（广播地址）
- LAN 上所有节点都能收到这条 query

2. B 单播 ARP Reply 给 A

- 告诉 A：“这个 IP 对应的 MAC 是我：B_MAC”
- reply 帧的目的 MAC 就是 A 的 MAC（unicast）

3. A 把 (B_IP → B_MAC) 缓存到 ARP table

- 课件强调：ARP 表是 **soft state**：会超时消失，除非刷新
- 表项形如 <IP, MAC, TTL>，TTL 典型 ~20 min  chapter_6_notes
- ARP 是 **plug-and-play**：不需要管理员手工配表  chapter_6_notes

你现在可以把 ARP 记成一句话：

ARP = “在一个 LAN 内，用广播问 IP 对应哪个 MAC，再把答案缓存起来”。

 chapter_6_notes

停一下：两道超关键自检（你答完我就继续“跨LAN经由路由器”的 walkthrough）

1. ARP Query 为什么必须用 广播 MAC (FF-FF-...)，而不是直接单播？（一句话）

 chapter_6_notes

2. ARP 表为什么说是 **soft state**？TTL 到了会发生什么？ chapter_6_notes

Sources

1. 因为不知道目的地址在问所有人 2. 会超时消失

对，完全正确 

那我们继续课件里最重要、也最容易混的部分：**跨子网发送时，MAC 地址怎么填？ARP 问的是谁？路由器到底改了什么？**（也就是你之前问过的 next-hop 相关问题）。

 chapter_6_notes

6-50~6-51：跨 LAN 的寻址 walkthrough（A 发给 B，但 B 不在同一 LAN）

课件设定通常是这样：  chapter_6_notes

- A 在子网 1 (LAN1)，IP = A_IP，MAC = A_MAC
- 路由器在 LAN1 上的接口：IP = R1_IP，MAC = R1_MAC
- 路由器在 LAN2 上的接口：IP = R2_IP，MAC = R2_MAC
- B 在子网 2 (LAN2)，IP = B_IP，MAC = B_MAC

关键原则（先背一句）

IP 头的目的地址一直是 B_IP（端到端目标不变），但每一跳帧的目的 MAC 都是“下一跳接口”的 MAC。  chapter_6_notes

Step-by-step：A 真正怎么发出去

Step 1：A 决定“下一跳是谁”（默认网关）

A 一看：B_IP 不在本子网（用子网掩码判断）。

所以 A 不是直接找 B 的 MAC，而是把包交给 **默认网关**（路由器在本 LAN 的接口）。

 chapter_6_notes

Step 2：A 用 ARP 找到网关接口的 MAC（不是找 B 的）

如果 A 的 ARP 表里没有 R1_IP → R1_MAC：

- A 广播 ARP：Who has R1_IP？

- 路由器的 LAN1 接口回 ARP reply: R1_MAC
- A 缓存表项

Step 3: A 封装第一跳 frame 发给路由器

- IP datagram: src IP = A_IP , dst IP = B_IP (注意: dst 仍然是 B)
- Ethernet frame:
 - src MAC = A_MAC
 - dst MAC = R1_MAC (下一跳: 路由器)  chapter_6_notes

A 把这个 frame 发到 LAN1, 交换机按 dst MAC 转发, 最终到路由器接口。

路由器收到后做什么 (这一步最容易混)

路由器在 LAN1 接口收到 frame:

1. 拆帧: 检查 CRC, 取出里面的 IP datagram
2. 查路由表: 决定下一跳要从 LAN2 接口出去
3. 重新封装成一个新的 frame 在 LAN2 上发:
 - IP datagram 里: src IP = A_IP , dst IP = B_IP (仍然不变)
 - 新 frame:
 - src MAC = R2_MAC (路由器在 LAN2 的接口 MAC)
 - dst MAC = B_MAC (最终主机的 MAC)
 - 如果路由器不知道 B_IP → B_MAC , 它也会在 LAN2 上 ARP 一次。

然后 B 收到 frame, 拆帧得到 datagram, 上交给 IP 层。

你必须记住的“考试句”

1. ARP 只在一个 LAN 内工作: 广播不会跨路由器转发。
2. 跨子网时, 主机 ARP 的对象是默认网关的 MAC, 不是远端主机的 MAC。

 chapter_6_notes

3. 每过一跳, frame 的 MAC 地址都会变; IP 目的地址一般不变。  chapter_6_notes

小停一下 (两题, 确保你真的不混)

1. A 要访问远端 B: A 的第一跳 frame 里 **dst MAC** 应该填 B_MAC 还是 R1_MAC ? 为什么?
2. 路由器转发时会改哪些字段:
 - MAC 源/目的?
 - IP 源/目的? (先按“通常情况”回答)

你答完我就继续课件后面的 **Ethernet (交换机、自学习、碰撞域/广播域)**。

Sources



1. bmac2. mac源会改; ip源改目的不改

已思考 29s >

你刚才那两题里 (1) 和 (2) 都需要纠正一下, 否则后面 Ethernet/switch 会一直串不起来:

先纠正: 跨 LAN 时第一跳的 dst MAC 不是 B_MAC

当 A 要发给不同 LAN 的 B 时:

- A 发出去的第一跳帧: dst MAC = 默认网关/路由器在 A 所在 LAN 的接口 MAC (因为 ARP 只能在本 LAN 内广播问答案; 你也不可能直接在 LAN1 上拿到 LAN2 里 B 的 MAC)。
- 路由器转发到 B 所在 LAN 的那一跳: 才会封装成 dst MAC = B_MAC。课件专门给了这张图: 路由器 R 转发时, IP src/dst 仍是 A→B, 但它创建的新以太帧的 MAC dest 是 B 的 MAC。  chapter_6_notes

所以你第 1 问应该答: R1_MAC (默认网关的 MAC)。

再纠正: 路由器转发时 IP 源/目的通常不改

在“通常情况” (不谈 NAT):

- MAC 源/目的: 每一跳都会改 (因为重新封装新 frame)  chapter_6_notes
- IP 源/目的: 一般不改 (仍是 A_IP→B_IP)  chapter_6_notes
(实际实现里会改 TTL 等, 但课件这页强调的是 src/dst 保持 A→B。)

继续过课件：Ethernet（6-56～6-61）

6-56 以太网为什么“统治”

便宜、简单、一路提速（10 Mbps 到 10 Gbps）。  chapter_6_notes

6-57 物理拓扑：bus vs star

- **bus**（老式同轴总线）：所有节点在同一 collision domain，会互相碰撞
- **star**（现在主流）：中心是交换机，节点之间不碰撞（每条链路更独立）

 chapter_6_notes

6-58～6-59 Ethernet frame 结构（你要会认字段）

发送端网卡把 IP datagram 封装进 Ethernet frame：  chapter_6_notes

- **preamble（8 bytes）**：同步用（7 字节 10101010 + 1 字节 10101011）
 chapter_6_notes
- **dest / source MAC（各 6 bytes）**：目的/源网卡地址
 - 收到帧时：目的 MAC 匹配自己或是广播地址才上交，否则丢弃  chapter_6_notes
- **type**：上层协议类型（大多是 IP）  chapter_6_notes
- **data(payload)**：承载 IP datagram
- **CRC**：差错检测；错了就丢帧  chapter_6_notes

6-60 Ethernet 的服务特性（很考概念）

- **connectionless**：网卡之间不握手
- **unreliable**：接收方 NIC 不发 ACK/NAK；丢了就丢了，恢复靠更高层（如 TCP）
 chapter_6_notes
- **MAC 协议**：unslotted CSMA/CD + binary backoff（你前面刚学的 NIC 等待规则就是这里）

继续：Switches（6-63～6-69）

6-63 交换机是什么

交换机是 **链路层设备**：store-and-forward 以太帧、看 MAC 地址选择性转发；对主机透明；还能 plug-and-play + self-learning（无需手工配置）。  chapter_6_notes

6-64 为什么 switch 让“同时传输”成为可能

- 主机到交换机是 **专用链路**
- 交换机可以缓存帧
- **每条链路是一个 collision domain**，并且可以 full-duplex（某接口可同时收发）
=> $A \rightarrow A'$ 和 $B \rightarrow B'$ 可以并行、无碰撞。  chapter_6_notes

6-65~6-68 交换机表 & 自学习（这段要背成算法）

交换机维护一个表（像路由表）：

（host MAC，出口接口，timestamp/TTL）  chapter_6_notes

****自学习规则：****帧从某接口进来时，交换机把 **源 MAC** 记到表里：srcMAC → incoming interface。  chapter_6_notes

转发/过滤规则（课件 6-68 三步）：  chapter_6_notes

1. 记录：进入链路 + 发送方 MAC
2. 用 **目的 MAC** 查表
3. 若查到：
 - 如果目的就在同一入端口方向 → 丢弃（过滤）
 - 否则按表项指示的端口转发若查不到：**flood**（除入端口外全转发）

6-69 交换机互联也一样学

多台交换机串起来仍靠同一套 self-learning 工作，不需要“交换机路由协议”。

 chapter_6_notes

停一下（两道自检，答完我再继续 VLANs 或后面的 “day in the life”）

1. 为什么 star + switch 里说“没有碰撞”，但以太网帧里仍然有 CRC（差错检测）？
2. 交换机在学习表项时，用的是 **src MAC** 还是 **dst MAC**？为什么？

Sources



1. 除了碰撞，还有别的因素如物理环境因素也可能造成差错
2. src

对，两个都对 ✅（CRC 是为“传输比特错误”服务，不只是碰撞；交换机必须用 **src MAC** 才能从“帧从哪个口进来”学到回程路径。）

继续按课件走到下一块：**交换机 vs 路由器（6-70/6-71）**，这块是考试高频对比题。我讲完这一节就停给你提问。

6-70/6-71: Routers vs Switches（核心对比）

课件要你抓住的是：它们都在“转发”，但工作层次、依据、效果完全不同。

1) 工作层次不同

- **Switch（交换机）**：**链路层（L2）**设备，转发单位是 **frame**，主要看 **MAC 地址**。
- **Router（路由器）**：**网络层（L3）**设备，转发单位是 **IP datagram**，主要看 **IP 前缀/路由表**。

2) “自学习” vs “路由计算”

- 交换机：plug-and-play，自学习（从进入接口学习 src MAC→port）
- 路由器：需要路由算法/协议（OSPF/BGP 等）来决定下一跳（这就是你前面复习的 routing 那章）

3) 广播域（broadcast domain）差异是最关键的网络效应

- **交换机默认不会切广播域**：ARP 这种二层广播（FF:FF:FF:FF:FF:FF）会在同一 VLAN 内被 flood 到所有端口（除入端口）。
→ 所以一个“纯交换网络”通常仍是一个大的广播域。
- **路由器会切广播域**：路由器不转发二层广播，ARP 广播不会跨过路由器到另一个子网。
→ 所以“跨子网通信”必须走路由器。

你可以背一句：

Switch 扩大二层连通性；Router 隔离广播、连接不同 IP 子网。

4) (常考延伸) 碰撞域/广播域一句话总结

- 碰撞域：现代交换式以太网里，每条交换机端口链路基本是一个碰撞域（甚至 full-duplex 下几乎不碰撞）。
- 广播域：交换机不隔离，路由器隔离。

停一下：两道高频选择题式自检

1. 你在大二层网络里，ARP 广播包会不会被交换机转发到所有端口？会不会穿过路由器？（两句回答）
2. “交换机也能转发到远端网络，所以它和路由器差不多”——这句话哪里错？

你答完我就继续课件后面：VLAN（如果有）或者 Link virtualization / MPLS（看你 slides 的顺序）。



1. 会，不会 2. no，交换机不能传到远端网络

1. 对
2. 你这句“交换机不能传到远端网络”作为直觉是对的，但更精确（也更像考试答案）要这样说：
 - 交换机当然能把帧转发到**远处的交换机**（很多跳），但前提是它们仍在**同一个二层广播域/VLAN**里；它只是在扩展一个 LAN。
 - 真正意义上的“远端网络/不同 IP 子网”（需要三层路由、隔离广播、做下一跳选择）**必须靠路由器（L3）**。

继续按课件往下：通常下一块会讲 VLAN（虚拟局域网）——它正好回答“交换机网络太大、广播太多怎么办”。我按 VLAN 的标准课件结构给你过一遍（讲完停）。

VLAN：为什么要 VLAN

在纯交换网络里：

- ARP / DHCP 等二层广播会被 flood，网络一大广播就多；
- 安全隔离也弱（同一二层里更容易互相看到）。

VLAN 的目标：在同一套物理交换机上，把二层网络逻辑切成多个“独立的广播域”。
同 VLAN 内二层可达；不同 VLAN 之间要通信必须经由 路由器/三层交换。

VLAN 怎么做：Access port vs Trunk port（最常考）

课件一般会区分两类端口：

1) Access port（接主机）

- 一个端口只属于一个 VLAN
- 主机发普通以太网帧（不带 VLAN tag）
- 交换机“知道这个端口属于 VLAN X”，就把该帧归到 VLAN X 的转发表/广播域里

2) Trunk port（交换机↔交换机/交换机↔路由器）

- 一根链路要承载多个 VLAN 的流量
- 所以帧在 trunk 上会带 VLAN tag（802.1Q 标签），里面有 VLAN ID，告诉对端“这是 VLAN 多少的帧”。

一句话：access 口不打标签，trunk 口打标签。

VLAN 带来的直接后果（务必记住）

- 广播被限制在 VLAN 内：VLAN A 的 ARP 广播不会跑到 VLAN B。
- 不同 VLAN 要互通：必须经过三层（router-on-a-stick 或 L3 switch），因为交换机不会在 VLAN 间做路由。

停一下：你答两个点，我就继续讲“VLAN 间通信怎么做（router-on-a-stick）”

1. 为什么 trunk 上一定要有 VLAN tag？（一句话）
2. VLAN 把“什么域”切开了：碰撞域还是广播域？

📄 🍷 🗨️ 📶 🔄 ...

?

没事，这两个点我直接讲清楚（这是 VLAN 的核心）。

1) 为什么 trunk 上一定要有 VLAN tag?

因为 trunk 一根物理链路要同时承载多个 VLAN 的流量。

- 如果不打 tag，对端交换机收到一帧时就不知道它属于 VLAN10 还是 VLAN20
- 一旦分不清，就没法把它放进正确的广播域/转发表里，也就等于 VLAN 失效了

所以 trunk 上要用 802.1Q tag 里携带 VLAN ID 来“标记这帧属于哪个 VLAN”。

(对比：access 口只接一个 VLAN，所以不需要 tag，交换机用端口配置就能判断 VLAN。)

2) VLAN 切开的是碰撞域还是广播域?

切开的是 广播域 (broadcast domain)。

- 碰撞域：现代交换式以太网里，本来每个交换机端口链路就基本独立（尤其全双工下几乎无碰撞），不是 VLAN 的主要目标。
- 广播域：VLAN 的核心就是让 ARP/DHCP 等二层广播 只在本 VLAN 内传播，减少广播风暴并做隔离。

继续：不同 VLAN 之间怎么通信 (router-on-a-stick 思想)

由于交换机不会跨 VLAN 转发二层帧，VLAN 间通信必须走三层路由。常见做法之一是：

- 交换机到路由器之间拉一条 trunk
- 路由器上配置多个子接口（每个子接口对应一个 VLAN，有各自的 IP 网关）
- VLAN10 的主机要去 VLAN20：先发给 VLAN10 的默认网关（路由器子接口），路由器再转发到 VLAN20，再由交换机在 VLAN20 内二层送达

你现在可以把它记成一句话：

VLAN 解决“二层隔离”，跨 VLAN 的“互通”交给路由器。

停一下，确认你通了（只答对/错）

- A. VLAN10 的 ARP 广播会被 trunk 转发到 VLAN20。
- B. VLAN 间通信时，帧在 trunk 上必须带 VLAN tag。
- C. 路由器转发 VLAN10→VLAN20 时，IP 目的地址会从 VLAN20 主机变成路由器接口。

你答完我就继续课件后续（通常是：MPLS / 链路虚拟化 / 数据中心交换等，看你的 slides 后面是什么）。



到6-72开始

已思考 13s >

好，我们从 **Link Layer 6-72** 开始：这部分是把前面学的所有层“串起来”，看一次完整的 Web 访问到底牵扯了哪些协议。  chapter_6_notes

6-72: Synthesis 的目标是什么

课件这页的意思是：你已经分别学过 **应用层、传输层、网络层、链路层**，现在做一个“合成题”——在一个看起来很简单的场景里，把每一层用到的协议都点出来、按顺序串起来：

 chapter_6_notes

场景：学生把笔记本连上校园网，然后请求/接收 `www.google.com` 。  chapter_6_notes

你可以提前在脑子里放一个“必经清单”：

1. 先拿到网络配置：IP/默认网关/DNS（DHCP）
2. 本地链路要能发出去：知道网关 MAC（ARP）
3. 把域名变成 IP（DNS）
4. 跟服务器建立连接（TCP 3 次握手）
5. 发请求收网页（HTTP）

6-73: 这张拓扑图你要读懂什么

这页给了一个具体的“互联网分段”示意图：  chapter_6_notes

- **school network: 68.80.2.0/24**（校园网子网；你的 laptop/browser 在这里）
- **Comcast network: 68.80.0.0/13**（ISP 网络，校园网的上游）
- **Google's network: 64.233.160.0/19**（Google 所在的网络）
- 右边标了一个具体的 **web server: 64.233.169.105**（这就是最终要连的服务器 IP）

- 图里还标了 **DNS server** (给你把 `www.google.com` 解析成上面这个服务器 IP 的)

📄 chapter_6_notes

这张图的“考点读法”是：

- 你的 laptop 一开始只在 **school network** 里，它要访问 Google 必然要先把包交给 **第一跳路由器**，再经过 ISP (Comcast) 跨网到 Google。
- **IP 层**看的是：从 `68.80.2.0/24` 走到 `64.233.160.0/19`。
- **链路层**看的是：每一跳都要封装成帧，MAC 地址每跳都会换（你之前已经学过这个原则）。 📄 chapter_6_notes

停一下，确认你能“读图”

1. 你能用一句话说清：为什么校园网这里写 `/24`，Comcast 写 `/13`？（提示：它们是不同的“网络前缀/规模”） 📄 chapter_6_notes
2. 在这张图里，你觉得 **DNS server** 更可能在校园网里还是在 ISP(Comcast) 里？你凭直觉猜一个就行（不需要100%确定）。

你答完我就从 **6-74/6-75：DHCP 获取地址**开始，按课件一步步把每个封装（DHCP→UDP→IP→Ethernet）串起来。 📄 chapter_6_notes

Sources



1. 不同规模的子网2. ?

已思考 15s >

好，从 **6-74 / 6-75** 正式开始走“day in the life”的第一步：**DHCP** 拿到三样东西：自己的 IP、第一跳路由器 IP、**DNS server** 的名字和 IP。 📄 chapter_6_notes

（顺便回答你刚才“DNS server 在哪？”：从后面的 6-77 看，这个 DNS query 会被路由进 Comcast 网络再到 DNS server，暗示 DNS server 很可能在 ISP/上游网络侧，而不是你本地那条以太网里。 📄 chapter_6_notes)

6-74: 刚接入校园网时, 为什么先跑 DHCP? 怎么发出去的?

课件说: 笔记本刚接入时需要三样配置:  chapter_6_notes

- 自己的 IP address
 - first-hop router (默认网关) 的 IP
 - DNS server 的名字和 IP
- 用 DHCP 来拿。

然后这一页强调“封装链条”一定要会画:  chapter_6_notes

- DHCP 报文 encapsulated in UDP
- UDP encapsulated in IP
- IP encapsulated in 802.3 Ethernet

关键细节: 以太网帧是广播发出去的 (因为你还不知道 DHCP server/路由器的 MAC), 目的 MAC 是 FF:FF:FF:FF:FF:FF, 所以整个 LAN 都会收到, 最终“跑 DHCP server 的路由器”会处理它。  chapter_6_notes

并且课件点出“解复用 (demux)”链条: 收到广播帧后

Ethernet → IP → UDP → DHCP, 一层层把 payload 交给上层协议。  chapter_6_notes

6-75: DHCP server 回 ACK, 客户端最终“知道了什么”

路由器 (DHCP server) 会生成 DHCP ACK, 里面包含:  chapter_6_notes

- 分配给客户端的 client IP
- 客户端的 first-hop router IP
- DNS server 的 name & IP

然后同样是封装发送回去 (DHCP→UDP→IP→Eth), 在交换机里转发 (交换机会自学习转发表), 客户端收到后再 demultiplex 上交 DHCP。最终课件一句话总结:  chapter_6_notes

Client now has IP address, knows name & addr of DNS server, IP address of its first-hop router.

小停一下 (两个超短自检)

1. DHCP request 为啥用以太网广播 FF:FF:FF:FF:FF:FF? (一句话)

 chapter_6_notes

2. DHCP ACK 里给了你哪“三样关键配置”？（按课件列） chapter_6_notes

你答完我就继续 **6-76：ARP（在 DNS/HTTP 之前）**，把“先 ARP 拿网关 MAC、再发 DNS query”的顺序串起来。

Sources



pause，前面的问题我不会你还没回我呢

已思考 13s >

对，你刚才卡住的是 **6-73 的问题 2：DNS server 更可能在校园网里还是在 ISP(Comcast) 里？**

按照课件后面 **6-77** 的描述：客户端的 DNS query 先从校园网到第一跳路由器，然后 **“forwarded from campus network into Comcast network ... routed ... to DNS server”**——也就是说 DNS server 在图里是“要通过 Comcast 网络路由过去”的那个节点。

所以这题的答案是：

- **更可能在 ISP/Comcast 网络侧（或至少不在你本地那条 LAN 上）**，因为需要从 campus network 路由进入 Comcast network 才能到 DNS server。

如果你 OK 了，我们再继续从 **6-74 DHCP 广播封装链条**往下走（DHCP→UDP→IP→Eth）。



继续

好，继续从课件 **6-76 开始：ARP**，把“DHCP 完了之后，为什么要先 ARP，再 DNS，再 TCP/HTTP”这条链路串起来（我只讲到 **6-77 DNS 拿到服务器 IP**，讲完停一下给你提问）。

6-76: DHCP 之后，为什么立刻出现 ARP?

DHCP 结束后，你已经知道三样东西：你的 IP、默认网关（first-hop router）的 IP、DNS server 的 IP。  chapter_6_notes

但注意：要把任何 IP 包发出本子网，第一跳都得交给默认网关。

而在以太网里发帧必须填 目的 MAC，所以你还缺一个关键东西：

默认网关接口的 MAC 地址（也就是 router 在你这条 LAN 上那块网卡的 MAC）。

 chapter_6_notes

于是 6-76 的动作是：

1. 客户端检查 ARP 缓存，没有网关表项
2. 客户端广播 **ARP query**：“谁是 first-hop router IP？把你的 MAC 告诉我”
3. 路由器回 **ARP reply**，客户端把（router IP → router MAC）缓存起来

 chapter_6_notes

关键点：这里 ARP 的对象是 **网关 MAC**，不是 DNS server 的 MAC，更不是 web server 的 MAC（因为它们都不在同一条 LAN 内）。  chapter_6_notes

6-77: DNS query 开始（拿到 www.google.com 的 IP）

有了网关 MAC 之后，客户端就能把 DNS 查询包发出去了。课件这页的“链条”是这样：

(1) 客户端发 DNS query：封装关系

- DNS query 封装进 UDP
- UDP 封装进 **IP datagram**（目的 IP = DNS server 的 IP）
- IP datagram 封装进 **Ethernet frame**（目的 MAC = 默认网关的 MAC）

注意这一跳的 frame：**dst MAC 是网关**，但 IP 的 **dst IP 是 DNS server**。

这正是“每跳 MAC 变、端到端 IP 不变”的具体体现。

(2) 路由器怎么转发

路由器收到帧后：

- 拆帧取出 IP 数据报
- 看目的 IP（DNS server）

- 把它路由进 Comcast 网络，再一路转发到 DNS server

(3) DNS server 回答：给出 web server IP

DNS server 最终回一个 **DNS reply**，里面包含 `www.google.com` 对应的 IP 地址（课件图里示例 web server 是 `64.233.169.105`）。客户端收到 reply 后，浏览器终于知道要连谁。

小停一下（确保你没混）

1. 在 6-77 里，客户端发 DNS query 的那一跳：
 - Ethernet **dst MAC** 是谁？
 - IP **dst IP** 是谁？（两句回答）
2. 为什么必须在 DNS 之前先做 6-76 的 ARP？（一句话）  chapter_6_notes

你答完我就继续下一张（通常是 **TCP 3-way handshake + HTTP request/response**，也就是“真正开始下载网页”）。

Sources



请你给我解释回答这个问题并继续

已思考 20s >

好，我先把刚才那两道自检题“标准回答 + 为什么”讲清楚，然后继续到 **6-78/6-79**（TCP 三次握手 + HTTP 请求/响应）。  chapter_6_notes

先解释你要回答的两个问题（对应 6-76/6-77）

1) 在 6-77 里客户端发 DNS query 的那一跳：

- Ethernet 的 **dst MAC** = 第一跳路由器（默认网关）在本 LAN 上的 MAC
- IP 的 **dst IP** = DNS server 的 IP  chapter_6_notes

为什么会出现“MAC 指向网关，但 IP 指向 DNS server”的组合？

- 因为 **MAC 地址只管同一条链路/同一 LAN 内“这一跳送给谁”**，跨子网必须先交给网关；
- 而 **IP 地址管端到端要到哪里**，你真正想找的是 DNS server，所以 IP 目的还是 DNS server。  chapter_6_notes

2) 为什么必须在 DNS 之前先做 6-76 的 ARP?

因为在以太网上你要发出那个“装着 DNS query 的帧”，必须填 **目的 MAC**；但你一开始只知道“网关的 IP”，不知道它的 MAC，所以必须先用 **ARP 广播问**到网关接口的 **MAC**，才能把帧送到路由器。  chapter_6_notes

继续课件：6-78 TCP connection carrying HTTP

DNS 回来后，你已经拿到了 `www.google.com` 对应的 **web server IP**（例子里是 **64.233.169.105**），接下来才能真的“连服务器”。  chapter_6_notes

6-78 这页讲的是：**要发 HTTP request**，客户端先打开到 web server 的 **TCP socket**，并开始三次握手：  chapter_6_notes

1. 客户端发送 **TCP SYN**（三次握手 step 1），跨域路由到 web server
2. web server 回 **TCP SYNACK**（step 2）
3. （标准 TCP）客户端再回 **ACK**（step 3），于是 **TCP connection established**（连接建立）

这页的“层层封装”你也要保持那个感觉：TCP 段被封装进 IP，再封装进以太网帧，一跳跳经过路由器转发。  chapter_6_notes

继续课件：6-79 HTTP request/reply

TCP 连接一旦建立，浏览器就把 HTTP 请求写进这个 TCP socket：  chapter_6_notes

- **HTTP request sent into TCP socket**
- 装着 HTTP request 的 **IP datagram** 被路由到 `www.google.com`
- web server 收到后，返回 **HTTP reply**（包含网页内容）
- 装着 HTTP reply 的 **IP datagram** 再被路由回客户端
- 最后网页显示出来（slide 里那个 finally!!!）  chapter_6_notes

停一下给你提问（两道快问）

1. 为什么课件在 6-78 强调“先 TCP connection, 再发 HTTP request”? (用一句话描述 TCP 对 HTTP 的作用)  chapter_6_notes
2. 在 6-79 里, “HTTP reply 被路由回客户端”说明了一个事实: IP 层是端到端的。你能用一句话说这意味着什么吗?

Sources



请你给我解释回答这个问题并继续

已思考 15s >

好, 我先把刚才留的那两个问题“标准回答 + 直觉解释”讲清楚, 然后继续到 6-80 / 6-81 结束本章。  chapter_6_notes

问题 1: 为什么 6-78 强调“先 TCP connection, 再发 HTTP request”?

因为在课件这个版本里, HTTP 是跑在 TCP 之上的: 浏览器要把 HTTP request “写进 TCP socket”, 前提是这个 socket 对应的 TCP 连接已经建立。

更直觉一点:

- TCP 提供 **可靠、按序的字节流** (丢了会重传、乱序会重排), 还带拥塞控制/流量控制;
- HTTP 把请求/响应当成一段段字节交给 TCP, 自己不需要去处理“丢包、乱序、重传”这些麻烦事。

所以课件才写得很明确: “to send HTTP request, client first opens TCP socket to web server”, 并进入三次握手流程 (SYN、SYNACK...)。  chapter_6_notes

问题 2: 6-79 里“HTTP reply 被路由回客户端”说明 IP 层端到端, 意味着什么?

意味着：路由器只负责把 IP 数据报一跳跳转发到“目的 IP”，它们不需要理解你里面装的是 HTTP 还是 DNS，也不需要维护应用层会话状态。  chapter_6_notes

在 6-79 的一句话里体现得很清楚：

- “IP datagram containing HTTP request routed to www.google.com” ↗
- “IP datagram containing HTTP reply routed back to client”  chapter_6_notes

也就是：只要 源/目的 IP 写对了，网络层会把去程送到服务器，把回程送回客户端；上层（TCP/HTTP）只是“乘客”，被 IP 这个“运输系统”端到端搬运。

继续课件：6-80 Chapter 6 Summary

这一页就是把 Chapter 6 的大块内容收束成三组：  chapter_6_notes

1. 链路层服务背后的原则

- error detection / correction（CRC、校验等）
- sharing broadcast channel: multiple access（ALOHA/CSMA/轮询/令牌/DOCSIS）
- link layer addressing（MAC、ARP）  chapter_6_notes

2. 具体技术实现

- Ethernet
- switched LANs, VLANs
- virtualized networks as a link layer: MPLS  chapter_6_notes

3. 综合：day in the life

- 把 DHCP → ARP → DNS → TCP → HTTP 串起来（你刚刚做的就是这件事）

继续课件：6-81 let's take a breath

这一页是“下课总结式”的话：  chapter_6_notes

- 你已经把协议栈从应用层一路走到链路层（除了 PHY）
- 已经有扎实的 networking principles & practice
- 后面还有很多拓展主题：wireless / multimedia / security / network management

 chapter_6_notes

停一下（最后两道检查，确保你对 6-72~6-79 的链路完全闭环）

1. **按顺序**写出这个场景里出现的协议/机制（只写关键词）：从“刚插网线/连上网”到“网页显示出来”。  chapter_6_notes
2. DNS query 那一跳里：**Ethernet 的 dst MAC 是谁？IP 的 dst IP 是谁？**（一句话）
 chapter_6_notes

你答完，如果你想复习“最容易考的点”，我可以帮你把本章高频考点做一张“考前速记清单”（不长，但很顶用）。

Sources



回答并解释这个问题

好，我把你刚才那两道“检查题”直接给出**标准答案 + 解释**（对应课件 6-72~6-79 的 day-in-the-life 流程）。  chapter_6_notes

1) 从“连上网”到“网页显示”按顺序出现哪些协议/机制？

按课件这条线，一般是：

1. **DHCP**：拿到
 - 自己的 IP
 - 默认网关（first-hop router）的 IP
 - DNS server 的 IP/名字

（DHCP 报文封装在 UDP/IP/Ethernet，且一开始用二层广播发出去）
2. **ARP**：把“默认网关的 IP → 默认网关的 MAC”解析出来并缓存（因为你要发以太网帧必须填目的 MAC）
3. **DNS (UDP)**：向 DNS server 发 query，把 `www.google.com` 解析成 web server 的 IP（DNS query/response 都封装在 UDP/IP 里）
4. **TCP 三次握手**：对 web server 的 IP 建立 TCP 连接（SYN / SYNACK / ACK）
5. **HTTP over TCP**：
 - 发 HTTP request
 - 收 HTTP reply（网页内容）
 - 浏览器渲染显示

一句话解释：DHCP 配地址 → ARP 找到第一跳 MAC → DNS 把域名变 IP → TCP 建连接提供可靠字节流 → HTTP 传网页内容。

2) DNS query 那一跳里：Ethernet 的 dst MAC 是谁？IP 的 dst IP 是谁？

答案：

- Ethernet dst MAC = 默认网关（first-hop router）在本 LAN 上的 MAC
- IP dst IP = DNS server 的 IP

解释：

- MAC 是“这一跳交给谁”：DNS server 不在你的本地 LAN 里，ARP 广播也不会跨路由器，所以你只能把帧先交给默认网关 → 目的 MAC 必须填网关的 MAC。
 - IP 是“端到端要到哪里”：你真正要查询的是 DNS server，所以 IP 目的地址仍然是 DNS server 的 IP，路由器会根据这个 IP 把包一路路由过去。
-

如果你愿意，我也可以把这一段整理成一张“考试作答模板”（看到 day-in-the-life 题就能直接套用的那种）。

Sources

